

**Zookeeper:** A Distributed Coordination Service for Distributed Applications.

A distributed coordination service is a system that helps manage and synchronize processes and data across multiple nodes in a distributed application. It provides a centralized, reliable way for distributed systems to handle tasks like configuration management, naming, synchronization, and leader election, ensuring consistency and fault tolerance.

### **Role of Zookeeper in Apache Kafka:**

In Apache Kafka, ZooKeeper plays a crucial role in managing and coordinating the Kafka brokers, including tracking their state, handling leader elections for partitions, and managing topic configurations. It acts as a centralized service, providing a consistent view of the Kafka cluster to all brokers.

## **Apache Kafka**

**Apache Kafka is an open-source distributed streaming system used for stream processing, real-time data pipelines, and data integration at scale.**

A **streaming system** delivers multimedia content (audio, video) over the internet in a continuous flow, allowing users to access it in real-time without needing to download the entire file first.

### **Advantages of apache kafka**

- 1.High Throughput:** Kafka can handle a large number of messages per second, making it suitable for high-volume data streams. It can process messages at a high velocity without significant performance degradation.
- 2. Low Latency:** Kafka offers low latency, meaning it can process messages with minimal delay. This is crucial for real-time applications where immediate responses are necessary.
- 3. Scalability:** Kafka's distributed architecture allows for easy scaling. You can add more brokers to the cluster to handle increasing data volumes and traffic without disrupting the system.
- 4. Fault Tolerance:** Kafka replicates data across multiple brokers, ensuring that data remains available even if some brokers fail. This provides a high degree of fault tolerance and reliability.
- 5. Durability:** Data in Kafka is persisted to disk, ensuring that it is not lost even if the system crashes. This provides a durable storage solution for your data streams.

**6. Consumer-Friendly:** Kafka provides features for consumers to manage their consumption of data streams, including the ability to rewind and replay messages.

### Disadvantages of kafka

- **Setup and Maintenance:** Kafka clusters can be complex to set up, configure, and maintain, requiring specialized knowledge and potentially significant resources.
- **Storage Costs:** Storing data indefinitely in Kafka can lead to high storage costs, especially with high-throughput data streams.
- **Data Retention:** Kafka is not designed for long-term storage. Data is eventually purged based on retention settings, which may not be suitable for all use cases.
- **Log Compaction:** While log compaction helps with storage, it can be complex to configure and may not be suitable for all scenarios.

### Topics

In Apache Kafka®, **topics** are the core abstraction for storing and processing data. Unlike traditional databases where data is stored in **tables** and updated with each new event, Kafka uses **logs** to record every event in sequence. This means that instead of replacing old values, each new event is simply **appended** to the log, preserving the entire history. For example, if you're monitoring a smart thermostat, every temperature reading is added as a new event in the log, allowing you to track changes over time.

- Each event in a topic is called a **message**, and it consists of three main parts:
- **A key**, which identifies the source of the event (like a thermostat ID or user ID).
- **A value**, which contains the actual data (like the temperature reading or the click action).
- **A timestamp**, which marks the exact time the event was produced.

## **Partition**

Whenever we will create topic we split it into different parts which is called partition.

### **Advantages of partitions:**

- **Scalability and Parallelism:**

Partitions enable horizontal scaling by distributing data across multiple Kafka brokers. This allows for parallel processing, as multiple producers can write to different partitions simultaneously, and multiple consumers within a consumer group can read from different partitions concurrently, leading to significantly higher throughput.

- **Fault Tolerance:**

Each partition can have multiple replicas distributed across different brokers. In the event of a broker failure, Kafka can serve data from the replicas on other brokers, ensuring data availability and reliability.

- **Load Distribution:**

By dividing a topic into partitions, Kafka can distribute the data processing workload across the cluster, preventing any single broker from becoming a bottleneck and ensuring efficient resource utilization.

- **Consumer Group Scaling:**

Partitions are the unit of parallelism for consumer groups. Each consumer in a consumer group can be assigned one or more partitions, allowing the consumer group to scale out and process data in parallel, thereby improving consumption speed and efficiency.

# OffSet

In Kafka, **offset** is a unique identifier assigned to each record (message) in a partition. Offsets are sequential integers that Kafka uses to maintain the order of messages within a partition. They help producers, consumers, and brokers track the specific position of a message in a partition.

- **Message Identification:**

Every message in a Kafka partition is assigned an offset, starting from 0 for the first message and incrementing for subsequent messages. This offset acts as a unique address for each message within its partition.

- **Consumer Position Tracking:**

Consumers in Kafka use offsets to keep track of their progress within each partition they are consuming from.

- **Current Offset:** This refers to the offset of the next message that will be delivered to the consumer. It automatically advances as the consumer receives messages.
- **Committed Offset:** This is the offset of the last message that a consumer has successfully processed and acknowledged. Consumers commit their offsets to an internal Kafka topic (typically `__consumer_offsets`) to ensure that if the consumer crashes or is rebalanced, it can resume processing from the last known committed position, preventing data loss or reprocessing of already processed messages.

## Consumer group

In Apache Kafka, a consumer group is a set of consumers that work together to read and process messages from one or more Kafka topics. Each consumer in the group is assigned a subset of the topic's partitions, allowing for parallel processing of data. When a consumer finishes processing a message from its assigned partition, it commits the offset (its position in the partition) to the Kafka broker, enabling fault tolerance and efficient scaling.

### Advantages of Consumer Group

- **Parallel Processing:**

Consumer groups enable multiple consumers to read from the same topic concurrently by dividing the topic's partitions among them.

- **Fault Tolerance:**

If a consumer in a group fails, the remaining consumers will automatically redistribute the failed consumer's partitions, ensuring no data is lost and the application continues to function.

- **Scaling:**

Adding or removing consumers from a group triggers a rebalancing of partitions, allowing for easy scaling of the processing capacity.

- **Consumer Offsets:**

Each consumer group tracks its progress through the topic by maintaining offsets for each partition. This allows consumers to pick up where they left off after a failure or when scaling up/down.

- **group.id:**

Consumers are grouped together by setting the group.id property in their configuration. Consumers with the same group.id belong to the same consumer group.

# Apache Kafka Interview Questions for Beginners

## What are topics in Apache Kafka?

### 2. Explain partitions in Apache Kafka

### 3. How are partitions distributed in an Apache Kafka cluster?

In an Apache Kafka cluster, partitions are distributed across brokers to achieve scalability, parallelism, and fault tolerance.

#### 1. Topic Partitioning:

- A Kafka topic is divided into one or more partitions. Each partition is an ordered, immutable sequence of records.
- The number of partitions for a topic is configured during topic creation.

#### 2. Partition Assignment to Brokers:

- Each partition resides on a specific Kafka broker.
- For fault tolerance, partitions can be replicated across multiple brokers, forming a set of in-sync replicas.
- One of these replicas is designated as the "leader" for that partition, handling all read and write requests. The other replicas are "followers" and simply replicate data from the leader.
- If the leader fails, Kafka automatically elects a new leader from the in-sync replicas.

#### 3. Producer Partitioning Strategies:

- **Key-based Partitioning:**

If a message has a key, Kafka uses a hashing function (e.g.,  $\text{hash}(\text{key}) \% \text{number\_of\_partitions}$ ) to determine which partition the message should be written to. This ensures that all messages with the same key go to the same partition, guaranteeing order for those specific messages.

- **Round-Robin Partitioning:**

If a message does not have a key (or the key is null), messages are distributed across partitions in a round-robin fashion. This ensures an even distribution of messages when ordering by key is not required.

- **Custom Partitioner:**

Producers can implement a custom Partitioner interface to define their own logic for assigning messages to partitions.

#### 4. Consumer Group Partition Assignment:

- Within a consumer group, partitions of a topic are distributed among the consumers in that group.
- Kafka's consumer group rebalancing mechanism ensures that each partition is consumed by only one consumer within a group at any given time.
- If consumers are added or removed from a group, or if partitions are added or removed from a topic, Kafka automatically rebalances the partition assignments among the active consumers in the group.

**What are consumers in Apache Kafka?**

**What are producers in Apache Kafka?**

**What is a broker in Apache Kafka?**

**What is a consumer group in Apache Kafka?**

**Mention the APIs provided by Apache Kafka.**

The main APIs are:

- **Producer API:**

This API allows applications to publish streams of records (messages) to one or more Kafka topics. It handles the details of sending data to the appropriate partitions within the Kafka cluster.

- **Consumer API:**

This API enables applications to subscribe to one or more Kafka topics and process the stream of records produced to them. Consumers can read messages from specific partitions and manage their consumption progress.

- **Streams API:**

This high-level API facilitates building real-time stream processing applications. It allows applications to act as stream processors, consuming input streams from topics, performing transformations or aggregations, and producing output streams to other topics.

- **Connect API:**

This API provides a framework for building and running reusable connectors that integrate Kafka with external data systems. Connectors can be used to import data from external sources into Kafka topics (source connectors) or export data from Kafka topics to external sinks (sink connectors).

- **Admin Client API:**

This API allows for programmatic management of Kafka clusters. It provides functionalities for tasks such as creating, deleting, and modifying topics, managing consumer groups, and inspecting cluster metadata.

**What is meant by ZooKeeper in Apache Kafka?**

**Can Kafka be used without a ZooKeeper?**

How is load balancing maintained in Kafka?

- **Partitioning:**

Kafka topics are divided into partitions, which are the fundamental unit of parallelism and data distribution. Each partition is an ordered, immutable sequence of messages. This allows for the distribution of data across multiple brokers and enables parallel processing by consumers.

- **Producer Load Balancing:**

- **Default Round-Robin:** By default, Kafka producers distribute messages to partitions within a topic in a round-robin fashion, aiming to spread the load evenly across available partitions.
- **Semantic Partitioning:** Producers can also specify a key for messages. Kafka then uses a hash of this key to determine the target partition, ensuring that all messages



with the same key are sent to the same partition. This is crucial for maintaining message order for a specific entity (e.g., a user ID).

- **Direct Broker Communication:** Producers send messages directly to the leader broker for the target partition, eliminating the need for an intermediate routing layer and improving efficiency.
- **Consumer Group Rebalancing:**

When multiple consumers belong to the same consumer group, they collectively consume messages from a topic's partitions. If a consumer joins or leaves the group, or if a partition is added or removed, Kafka automatically rebalances the partition assignments among the remaining consumers in the group to ensure continued consumption and distribute the workload.

- **Broker and Partition Rebalancing:**
  - **Partition Reassignment Tool:** Kafka provides tools to manually reassign partitions across brokers, which is useful for distributing load more evenly after adding new brokers or when a broker becomes overloaded.
  - **Automated Rebalancing (e.g., Self-Balancing Kafka):** Advanced features and tools, such as Confluent's Self-Balancing Kafka, can automatically monitor cluster load metrics and rebalance partitions to optimize performance, fill new brokers, or drain partitions from brokers being removed.
- **Replication:**

Kafka replicates partitions across multiple brokers to ensure data redundancy and fault tolerance. While primarily for availability, replication also contributes to load balancing by allowing consumers to fetch data from any available replica in case of a leader failure.

### **What is the maximum size of a message that can be received by Apache Kafka?**

The maximum size for a Kafka message, by default, is 1MB (megabyte). The size can be changed in the broker settings. However, Kafka is optimized to handle smaller messages of the size of 1KB.

### **Explain the retention period in an Apache Kafka cluster**

In Apache Kafka, the retention period defines how long messages are stored in a topic before they are eligible for deletion.

How it works:

- **Time-based retention:**

Kafka typically retains messages for a configurable time period, usually measured in hours, minutes, or milliseconds. The default retention period is 7 days (168 hours).

- **Segment-based deletion:**

Kafka organizes data into segments within a partition. When a segment's messages exceed the configured retention time, the entire segment is marked for deletion (or compaction, depending on the cleanup policy).

How long are messages retained in Apache Kafka?

In Apache Kafka, messages are retained for a configurable period, with the default being 7 days. This retention period is determined by the `log.retention.hours` setting, which defaults to 168 hours (7 days),

Here's a more detailed breakdown:

- **Default Retention:**

The default retention policy is 7 days, meaning messages older than 7 days are automatically deleted.

- **Configuration:**

You can adjust the retention period using `log.retention.hours` (or `log.retention.minutes` or `log.retention.ms` for more granular control).

- **Per-Topic Configuration:**

Kafka allows you to override the default retention policy and set different retention periods for specific topics.

- **Size-Based Retention:**

Besides time-based retention, Kafka also supports size-based retention, where messages are deleted when a partition's log reaches a certain size (specified by `log.retention.bytes`).

- **Log Compaction:**

Kafka also offers log compaction, a mechanism that retains only the latest value for each message key in a topic. This is useful for stateful use cases where you need to ensure you always have the most recent value.

- **Consumer Offsets:**

Kafka also manages consumer offsets, which track where consumers are in a topic's message stream. These offsets are typically retained for a shorter period than the messages themselves, but their retention is also configurable.

### **What is the role of the Partitioning Key?**

Messages are sent to various partitions associated with a topic in a round-robin fashion. If there is a requirement to send a message to a particular partition, then it is possible to associate a key with the message. The key determines

which partition that particular message will go to. All messages with the same key will go to the same partition. If a key is not specified for a message, the producer will choose the partition in a round-robin fashion.

### **When does `QueueFullException` occur in the Producer API?**

The `QueueFullException` occurs when the producer sends messages to the broker at a pace that the broker cannot handle. A solution here is to add more brokers to handle the pace of messages coming in from the producer

## **Explain the roles of leader and follower in Apache Kafka.**

Every partition in the Kafka server has one server that plays the role of a leader. The leader performs all the read and write data tasks for a partition. A partition can have no followers, one follower, or more than one follower. The job of the follower is to replicate the leader. In such a case, if there is a failure in the leader, then one of the followers can take on the leader's load.

## **What is the role of replicas in Apache Kafka?**

Replicas are the backups for partitions in Kafka. They are never actually read or written; rather, they are used to prevent data loss in case of failure.

## **What is the purpose of ISR in Apache Kafka?**

ISR - in-synch replicas refers to all the replicated partitions that are completely synced up with the leader. A replica has to be fully caught up with the leader within a configurable amount of time. By default, this time is 10 seconds. After this period of time, if a follower is not caught up with the leader, the leader will drop the follower from its ISR and writes will continue on the remaining replicas in the ISR. If the follower comes back, it will first truncate its log to the last point, which was checked, and then catch up on all the messages after the last checkpoint from the leader. Only when the follower fully catches up will the leader add it back to the ISR.

## **What is meant by partition offset in Apache Kafka?**

### **23. What is the importance of replication in Kafka?**

In Kafka, replication provides fault tolerance by ensuring that published messages are not permanently lost. Even if a node fails and they are lost on one node due to program error, machine error, or even due to software upgrades, then there is a replica present on another node that can be recovered.

## Explain fault tolerance in Apache Kafka.

Apache Kafka achieves fault tolerance primarily through data replication and a robust leader-follower mechanism for topic partitions.

### 1. Partition Replication:

- Kafka topics are divided into partitions, and each partition can have multiple copies, known as replicas.
- The number of replicas for a topic is defined by its "replication factor." A common production setting is a replication factor of 3, meaning each partition has three copies.
- These replicas are distributed across different Kafka brokers (servers) in the cluster.
- If one broker fails, the data for its partitions remains available on other brokers that host the replicas, preventing data loss and ensuring continued operation.

### 2. Leader-Follower Mechanism:

- For each partition, one replica is designated as the "leader," while the others are "followers."
- All read and write requests for a partition are handled exclusively by its leader.
- Followers continuously replicate data from the leader to stay synchronized.
- In the event of a leader failure, Kafka automatically elects a new leader from the set of "In-Sync Replicas" (ISRs). ISRs are followers that are fully caught up with the leader's data. This ensures that the new leader has all the committed data, preventing data loss during failover.

### 3. Acknowledgment (ACK) Mechanism:

- Producers can configure their acknowledgment settings (acks) to control the level of durability and fault tolerance.
  - acks=0: No acknowledgment from the broker. Fastest but highest risk of data loss.
  - acks=1: Leader acknowledges the write after storing the record. Faster but can lose data if the leader fails before replication.
  - acks=all (or -1): Leader waits for acknowledgment from all in-sync replicas. Safest, ensuring strong consistency and durability but with higher latency.

#### 4. Consumer Offset Tracking:

- Consumers track their progress by storing offsets, which indicate the last consumed message for each partition.
- This allows consumers to resume processing from where they left off if they crash or are restarted, ensuring no messages are lost or processed repeatedly.

#### **What is the best way to start the Kafka server?**

- Once you download the latest version of Apache Kafka, remember to extract it.
- To run Kafka, remember that your local environment must have Java 8+ installed on it.
- If you want to start the Kafka server, the following commands have to be run in order so that all the services can be started in the correct order:
- Start the ZooKeeper service:
- `$bin/zookeeper-server-start.sh config/zookeeper.properties`
- Open another terminal and run the following to start the Kafka broker service:
- `$ bin/kafka-server-start.sh config/server.properties`

#### **What is Geo-Replication in Kafka?**

- Geo-Replication in Kafka is a process by which you can duplicate messages in one cluster across other data centers or cloud regions. Geo-replication involves copying all the files and allows them to be stored across the globe if required.
- In Kafka, Geo-replication can be achieved by using Kafka's MirrorMaker Tool. Geo-replication is a way to ensure that the data is backed up.

#### **What is meant by multi-tenancy in Apache Kafka?**

- Multi-tenancy refers to the mode of operation of software where there are multiple instances of one or multiple applications operating in a shared environment independent from each other. The instances are said to be logically isolated but physically integrated. For a system to support multi-tenancy, the level of logical isolation must be complete, but the

level of physical integration may vary. Kafka can be said to be multi-tenant as it allows for configuring different topics for which data can be consumed or produced on the same cluster.

- 

**Explain the topic replication factor.**

- Topic replication factor refers to the number of copies of the topic that are present over multiple brokers. The replication factor should be greater than 1 for fault tolerance. In such cases, there will be a replica of the data in another broker from where the data can be retrieved if necessary.

**Differentiate between partitions and replicas in a Kafka cluster.**

- ¶ In Kafka, topics are divided into parts which are called partitions. Partitions allow one or more consumers to read data from servers in parallel. Read and write responsibility for one particular partition is managed on one server, called the leader for that partition. A cluster may have zero or more followers in which replicas of the data will be created. Replicas are merely copies of the data in a particular partition. The followers do not have to read or write the partitions separately; rather, they just copy the leader.
- ¶ Partitions in Kafka are used to increase throughput. Replicas ensure fault tolerance.

## **Mention some real-world use cases of Apache Kaka.**

- **Real-time Analytics and Monitoring:**

Kafka is used to collect and process large volumes of real-time data for analytics, such as website activity tracking, application performance monitoring, and business intelligence dashboards. Examples include LinkedIn's real-time analytics for user engagement and New Relic's use for collecting monitoring data.

- **Log Aggregation:**

It serves as a centralized system for collecting, storing, and processing logs from various applications and servers, facilitating easier debugging, troubleshooting, and security analysis. Uber's log aggregation system is a prominent example.

- **Event Sourcing and Microservices Communication:**

Kafka acts as a durable event log for event-driven architectures, enabling microservices to communicate asynchronously and reliably by publishing and subscribing to events. Walmart uses Kafka for event sourcing in its inventory management system.

- **Data Integration and ETL Pipelines:**

Kafka Connect, a component of Kafka, allows for seamless integration with various data sources and sinks, enabling real-time data movement and transformation for ETL (Extract, Transform, Load) processes. Expedia utilizes Kafka Connect for data integration across its systems.

- **Stream Processing:**

Kafka provides the foundation for building real-time stream processing applications using frameworks like Kafka Streams, Apache Flink, or Apache Spark Streaming, allowing for immediate insights and actions based on incoming data. Netflix's recommendation engine is a well-known example leveraging Kafka for stream processing.

- **Financial Services:**

In the financial sector, Kafka is used for real-time fraud detection, transaction monitoring, risk management, and building real-time trading platforms due to its low latency and high throughput.



- **Internet of Things (IoT):**

Kafka is ideal for ingesting and processing vast amounts of data generated by IoT devices and sensors, enabling real-time monitoring, predictive maintenance, and smart city applications. Tesla uses Kafka for managing data from its connected vehicles.

## **Why is Apache Kafka preferred over traditional messaging techniques?**

**Mention some of the system tools available in Apache Kafka.**

**Mention some benefits of Apache Kafka.**

## **What method does Apache Kafka use to connect with clients and servers?**

Apache Kafka uses a basic, high-performance, language-agnostic TCP protocol to facilitate communication between clients and servers. Backward compatibility exists between this protocol and its predecessor.

## **Define the role of Kafka Streams API and Kafka Connector API.**

- Streams API enables an application to work as a stream processor by efficiently changing input streams into output streams. Streams API is responsible for receiving input streams from one or more topics and sending output streams to one or more output topics.
- Connector API Connects Kafka topics to applications. The connector API enables the execution and building of reusable producers or consumers that link Kafka topics to pre-existing applications or data systems.

## **What is meant by the Replication Tool?**

- A replication tool is software that automatically copies and synchronizes data between multiple systems, databases, or storage environments. It ensures data consistency, high availability, and supports disaster recovery by enabling real-time or near-real-time updates. Essentially, it's used to create and maintain multiple copies of data across different locations, ensuring data accessibility and reliability.

**Core Function:**

- **Data Copying and Synchronization:**

- Replication tools copy data from a source system (like a database) to one or more target systems.

- **Ensuring Consistency:**

- They maintain consistency between the source and replica(s), meaning changes in the source are reflected in the replicas.

- **High Availability and Disaster Recovery:**

- Replication provides redundancy. If one system fails, another replica can take over, ensuring continuous operation and minimizing downtime.

### Common Use Cases:

- **Database Replication:**

- Creating copies of databases for different purposes, such as backups, reporting, or failover.

- **Data Warehousing:**

- Replicating data into a data warehouse for analytics and business intelligence.

- **Disaster Recovery:**

- Having a backup database in a separate location to quickly recover from a disaster.

- **Distributed Systems:**

- Enabling multiple systems to work with the same data.
- Examples of Replication Tools:
  - Hevo Data: A platform for real-time ELT and automated data integration.
  - Fivetran: An ETL platform with pre-built connectors for various data sources.
  - AWS DMS: A service for replicating data to and from AWS services.
  - Qlik Replicate: A tool for both real-time and bulk data movement.
  - Estuary Flow: A platform focused on real-time data replication and integration.
  - Airbyte: An open-source data integration platform.

**Is it possible to add partitions to an existing topic in Apache Kafka?**

- Apache Kafka provides the alter command to change a topic behavior and modify the configurations associated with it. The alter command can be used to add more partitions.
- The command to increase the partitions to four is as follows:
- `./bin/kafka-topics.sh --alter --zookeeper localhost:2181 --topic my-topic --partitions 4`

### **What is the optimal number of partitions for a topic?**

- The optimal number of partitions a topic should be divided into must be equal to the number of consumers.

### **What is the Kafka MirrorMaker?**

- The Kafka MirrorMaker is a stand-alone tool that allows data to be copied from one Apache Kafka cluster to another. The Kafka MirrorMaker will read data from topics in the original cluster and write the topics to a destination cluster with the same topic name. The source and destination clusters are independent entities and can have different numbers of partitions and varying offset values.

## 44. What is the role of the Kafka Migration Tool?

- The Kafka Migration tool is used to efficiently move from one environment to another. It can be used to move existing Kafka data from an older version of Kafka to a newer version.

## 47. What are name restrictions for Kafka topics?

According to Apache Kafka, there are some legal rules to be followed to name topics, which are as follows:

- The maximum length is 255 characters (symbols and letters). The length has been reduced from 255 to 249 in Kafka 0.10
- . (dot), \_ (underscore), - (hyphen) can be used. However, topics with a dot (.) and underscore ( \_) could cause some confusion with internal data structures, and hence, it is advisable to use either but not both.

## How can load balancing be ensured in Apache Kafka when one Kafka fails?

Kafka's distributed nature, combined with its replication, leader election, and consumer group rebalancing mechanisms, ensures that load balancing is maintained and data processing continues seamlessly even in the event of a broker failure.

- **Replication:**

Kafka partitions are replicated across multiple brokers. When a broker fails, the replicas on other brokers ensure **data availability**.

- **Leader Election:**

Each partition has a designated "leader" replica that handles all read and write requests for that partition. If the leader broker fails, Kafka automatically elects a new leader from the in-sync replicas, ensuring continuous operation.

- **Consumer Groups:**

Consumers in Kafka operate within consumer groups, where each consumer within a group is assigned a subset of partitions to consume from.

- **Automatic Rebalancing:**

When a consumer or broker fails, or a new consumer joins the group, Kafka triggers a rebalance. This process redistributes the partitions among the remaining active consumers in the group, ensuring that all partitions are consumed and the workload is evenly distributed.

## **Where is the meta-information about topics stored in the Kafka cluster?**

Currently, in Apache Kafka, meta-information about topics is stored in the ZooKeeper. Information regarding the location of the partitions and the configuration details related to a topic are stored in the ZooKeeper in a separate Kafka cluster.

## **How can large messages be sent in Apache Kafka?**

Sending large messages in Apache Kafka can be achieved through several methods, each with its own trade-offs:

- **Adjusting Kafka Configuration:**

- **Broker Settings:** Increase `message.max.bytes` (broker-wide) and `replica.fetch.max.bytes` (for inter-broker replication) to accommodate larger messages.
- **Producer Settings:** Set `max.request.size` to allow producers to send larger payloads.
- **Consumer Settings:** Adjust `fetch.max.bytes` and `max.partition.fetch.bytes` to enable consumers to fetch and process larger messages.
- **Topic-Specific Settings:** Configure `max.message.bytes` at the topic level if different topics require different message size limits.

- **External Storage and Reference-Based Messaging:**

- Store the large message in an external data store (e.g., Amazon S3, HDFS, Google Cloud Storage, a database).
- Send a small Kafka message containing a reference or pointer (e.g., a URL or ID) to the externally stored data.

- The consumer then retrieves the large message from the external store using this reference.
- This approach offloads the storage burden from Kafka and is suitable for very large messages (e.g., GB-sized files).
- **Message Compression:**
  - Enable compression on the producer side using the `compression.type` configuration (e.g., `gzip`, `snappy`, `lz4`, `zstd`).
  - This can significantly reduce the actual size of the data sent over the network and stored in Kafka, especially for highly compressible data.
- **Message Chunking/Splitting:**
  - At the producer, split the large message into smaller chunks that adhere to Kafka's message size limits.
  - Send each chunk as a separate Kafka message, ensuring they are ordered (e.g., by using the same partition key and including sequence numbers).
  - At the consumer, reconstruct the original large message by reassembling the chunks in the correct order.
  - This method keeps the data within Kafka but adds complexity to both the producer and consumer logic.

The optimal approach depends on the specific use case, message size, performance requirements, and existing infrastructure. For extremely large messages, external storage is often the most practical solution, while for moderately large messages, configuration adjustments and compression can be effective. Chunking is a viable option when keeping the data entirely within Kafka is a priority.

## **Explain the scalability of Apache Kafka.**

This is primarily achieved through:

- **Partitioning:**

Topics in Kafka are divided into multiple partitions. Each partition is an ordered, immutable sequence of records. These partitions can be distributed across different brokers (servers) within the Kafka cluster. This allows for parallel processing of data, as producers can write to different partitions concurrently and consumers can read from different partitions in parallel.

- **Distributed Architecture:**

A Kafka cluster consists of multiple brokers. By adding more brokers to the cluster, the system can linearly scale its capacity and throughput. This horizontal scaling allows Kafka to handle increasing data volumes and processing demands by simply adding more hardware resources.

- **Replication for Fault Tolerance and Scalability:**

Partitions can have multiple replicas, which are copies of the partition data stored on different brokers. This replication ensures fault tolerance, as data remains available even if a broker fails. Additionally, replicas can be used for read scaling, allowing consumers to read from a replica if the leader is busy or unavailable, though typically reads are from the leader.

- **Consumer Groups:**

Kafka's consumer groups enable multiple consumers to work together to consume messages from a topic in a scalable and fault-tolerant manner. Within a consumer group, each partition is assigned to only one consumer instance, allowing for parallel consumption across the partitions of a topic. As more consumers are added to a group, Kafka automatically rebalances the partition assignments, distributing the workload more efficiently.

## **What is meant by Kafka Connect?**

Kafka Connect streamlines the process of integrating Kafka with other systems. It makes it simple to quickly define connectors that move large data sets in and out of Kafka. Kafka Connect can ingest entire databases or collect metrics from all your application servers into Kafka topics, making the data available for stream processing with low latency. An export connector can deliver data from Kafka topics into secondary indexes like Elasticsearch, or into batch systems—such as Hadoop for offline analysis.

## **Explain message compression in Apache Kafka**

Message compression in Apache Kafka is a feature that allows producers to compress batches of messages before sending them to Kafka brokers. This compressed batch is then stored on the broker's disk in its compressed format and delivered to consumers in the same compressed state.

Benefits of Message Compression:

- **Reduced Network Bandwidth:**

Sending compressed data significantly reduces the amount of data transferred over the network between producers and brokers, and then between brokers and consumers.

- **Improved Throughput:**

Less data transfer leads to faster data movement and higher overall throughput in the Kafka cluster.

- **Lower Disk Utilization:**

Storing messages in a compressed format on the brokers reduces the amount of disk space required.

- **Reduced Latency:**

Faster data transfer contributes to lower end-to-end latency for message

## **Disadvantages of message compression**

- **Increased CPU Usage:**

Both producers and consumers require processing power to compress and decompress messages, respectively. This can become a bottleneck if the system is already CPU-bound.

- **Potential Latency Increase:**

Optimal compression often requires waiting for a sufficient number of messages to form a batch. This can introduce latency in message delivery.



## **Explain producer batch in Apache Kafka.**

producer batching is a mechanism employed by the Kafka producer to group multiple messages into a single request before sending them to the Kafka broker. This process significantly improves throughput and efficiency by reducing the overhead of individual network requests.

Instead of sending each message individually as soon as it's produced, the Kafka producer accumulates messages in an in-memory buffer, forming a "batch." This batch is then sent to the broker as a single unit.

Key Configuration Parameters for Batching:

- `batch.size`:

This parameter defines the maximum size in bytes that a batch can reach before it is sent. Once the accumulated messages within a batch reach this size, the batch is immediately sent, regardless of the `linger.ms` setting.

- `linger.ms`:

This parameter specifies the maximum amount of time, in milliseconds, that the producer will wait for more messages to accumulate in a batch before sending it. If the `batch.size` is not met within this time, the batch will still be sent. A default value of 0 means messages are sent immediately without waiting for batching.

Benefits of Producer Batching:

- **Increased Throughput:**

Sending fewer, larger requests reduces network overhead and improves the overall rate at which messages are sent to Kafka.

- **Improved Compression Efficiency:**

Larger batches often lead to better compression ratios, saving network bandwidth and storage space on the brokers.

- **Reduced CPU Utilization:**

Fewer requests mean less processing overhead for both the producer and the broker.

**Explain how Apache Kafka provides security.**

- **Authentication:**

This mechanism verifies the identity of clients (producers and consumers) and brokers attempting to connect to the Kafka cluster. Kafka supports various authentication methods, including:

- **SSL/TLS:** Secure Socket Layer (SSL) and Transport Layer Security (TLS) provide mutual authentication using certificates, ensuring that both clients and brokers can verify each other's identities.
- **SASL (Simple Authentication and Security Layer):** SASL offers a framework for integrating various authentication mechanisms, such as Kerberos, Plain, SCRAM, and OAUTHBEARER.

- **Authorization:**

Once authenticated, authorization determines what actions a client is permitted to perform. Kafka utilizes Access Control Lists (ACLs) to manage these permissions. ACLs define granular access rights for principals (users or applications) on specific Kafka resources, such as:

- **Topics:** Controlling read, write, or create access to topics.
- **Consumer Groups:** Managing consumer group operations.
- **Cluster Operations:** Restricting administrative actions on the cluster itself.

These ACLs are typically stored in Apache ZooKeeper, which also requires its own security measures.

- **Encryption:**

Kafka offers encryption for data in transit to protect against eavesdropping and tampering.

- **SSL/TLS Encryption:** This encrypts the network communication between clients and brokers, as well as inter-broker communication within the cluster. This ensures that data exchanged over the network remains confidential.

- **Auditing (via external tools):**

While Kafka itself doesn't provide built-in auditing features, its integration with external security tools and logging mechanisms allows for monitoring and auditing of security-related events, such as authentication attempts, authorization failures, and data access patterns. This helps in maintaining a comprehensive security posture and detecting potential breaches.

## **Can a consumer read more than one partition from a topic?**

Yes, if the number of partitions is greater than the number of consumers in a consumer group, then a consumer will have to read more than one partition from a topic.

For example, if a topic has 10 partitions and a consumer group has 5 consumers, each consumer will likely be assigned 2 partitions to read from. This allows for efficient resource utilization and scaling, as a single consumer can handle the load from multiple partitions if needed.

It is important to note that while a single consumer can read from multiple partitions, the message ordering guarantee only applies within each individual partition. The order of messages across different partitions consumed by the same consumer is not guaranteed.

## **Mention some use cases where Apache Kafka is not suitable.**

❑ Kafka is built to handle high volumes of data. If the requirement is to process only a small number of messages per day, a traditional messaging system would be more suitable.

❑ Kafka has a streaming API, but it is not suitable for performing data transformation operations. Kafka is to be avoided for ETL (extract, transform, load) jobs.

❑ In cases where a simple task queue is needed, there are better alternatives like the RabbitMQ.

❑ Kafka is not good if long-term storage is required. It supports saving data only for a specified retention period and not longer than that.

## **Define consumer lag in Apache Kafka.**

Consumer lag refers to the lag between the Kafka producers and consumers. Consumer groups will have a lag if the data production rate far exceeds the rate at which the data is getting consumed. Consumer lag is the difference between the latest offset and the consumer offset.

## **What do you know about log compaction in Kafka?**

Log compaction in Kafka is a mechanism that allows Kafka to retain the most recent version of a message for each key in a topic, effectively "compacting" the log by removing older, duplicate messages. This ensures that consumers can always retrieve the latest state of a key's value, even if multiple updates have occurred. It's a way to manage data within a topic based on keys, rather than just time-based or size-based retention.

How it works:

1. **1. Key-based retention:**

Log compaction relies on the keys associated with messages. Each key is treated as a unique identifier for a specific piece of data.

2. **2. Latest message retention:**

For each key, Kafka ensures that the most recent message (based on the message's offset or timestamp) is retained. Older messages with the same key are considered eligible for deletion.

3. **3. Log cleaning process:**

Kafka periodically cleans the log, identifying and removing obsolete messages. This process is asynchronous and happens in the background.

4. **4. Tombstone messages:**

If a message with a null value (a "tombstone" message) is produced, it indicates that the corresponding key should be deleted. This is a way to explicitly remove data from the compacted log.

Benefits of Log Compaction:

- **Storage efficiency:**

By removing older, duplicate messages, log compaction reduces the overall storage space used by Kafka.

- **Improved consumer performance:**

Consumers can retrieve the latest state of data quickly, as they don't need to traverse through potentially large numbers of older messages.

- **Data consistency:**

Log compaction ensures that consumers always see the latest value for a given key, even after multiple updates.

- **Reduced load on Kafka brokers:**

By reducing the size of the logs, compaction can also reduce the load on Kafka brokers, improving overall performance.

When to use it:

- **Stateful data:**

When you need to maintain the current state of data for a given key, like user profiles, inventory levels, or configuration settings.

- **Limited history:**

When you don't need to retain the entire history of messages, only the most recent one.

- **Efficient data retrieval:**

When you need to efficiently retrieve the latest value for a specific key from a topic.

### **What is meant by cluster id in Kafka?**

In Apache Kafka, a Cluster ID is a unique identifier assigned to a Kafka cluster. This ID serves as a crucial mechanism for ensuring the integrity and consistency of the cluster.

Here's a breakdown of its significance:

- **Cluster Identification:**

The Cluster ID uniquely identifies a specific Kafka cluster, distinguishing it from other Kafka deployments.

### **Why is the Kafka broker said to be “dumb”?**

In Apache Kafka, the term "dumb broker, smart consumer" refers to the architectural design where the Kafka broker primarily focuses on storing and delivering messages, while the consumers are responsible for managing their consumption state and processing logic. This contrasts with message brokers like [RabbitMQ](#), which are often described as having "smart brokers" that handle more complex tasks like routing and message delivery based on consumer subscriptions.

The Kafka broker doesn't track which messages a consumer has processed. It only stores messages for a configured retention period and then deletes them, regardless of whether they've been consumed.

### **What causes OutOfMemoryException?**

OutOfMemoryException can occur if the producers are sending large messages or if there is a spike in the number of messages wherein the producer is sending messages at a rate faster than the rate of downstream processing. This causes the message queue to fill up, taking up memory.

### **How can Kafka retention time be changed at runtime?**

#### **Explain the graceful shutdown in Kafka**

Graceful shutdown in Apache Kafka refers to the controlled and orderly process of stopping a Kafka broker, ensuring that ongoing operations are completed, data is persisted, and leadership roles for partitions are transferred to other available replicas before the broker fully shuts down. This minimizes data loss and service disruption during planned outages, maintenance, or configuration changes.

### **Can the number of partitions for a topic be reduced?**

No, Kafka does not allow reducing the number of partitions for a topic. The partitions can only be increased but not decreased.

## **Explain customer serialization and deserialization in Kafka**

In Kafka, serialization and deserialization are essential processes for handling data because Kafka brokers store and transmit data as raw bytes.

### **Serialization**

Serialization is the process of converting a structured data object (e.g., a Java object, a Python dictionary, a JSON string) into a sequence of bytes that can be transmitted over a network or stored on disk. When a Kafka producer sends a message, the key and value of that message are serialized into byte arrays before being sent to the Kafka broker.

- **Purpose:**

To convert application-specific data formats into a common, transportable format (bytes) that Kafka can handle.

- **Examples:**

- Converting a Java String to a byte array using `StringSerializer`.
- Converting a custom Java object to a byte array using Avro, Protobuf, or a custom serializer.

### **Deserialization**

Deserialization is the reverse process of serialization. It involves converting a sequence of bytes back into a structured data object that an application can understand and process. When a Kafka consumer receives a message from a Kafka broker, the byte arrays representing the key and value are deserialized into their original data types.

- **Purpose:**

To transform the raw bytes received from Kafka back into meaningful data objects for the consuming application.

- **Examples:**

- Converting a byte array back to a Java String using `StringDeserializer`.
- Converting a byte array back to a custom Java object using Avro, Protobuf, or a custom deserializer.

Why are they necessary in Kafka?

- **Data Storage and Transmission:**

Kafka brokers are designed to store and transmit data efficiently as byte arrays, regardless of the original data type. This provides flexibility and allows Kafka to handle diverse data formats.

- **Interoperability:**

Serialization and deserialization enable different applications written in various programming languages to produce and consume data from the same Kafka topics, as long as they agree on the serialization format.

- **Data Integrity:**

Proper serialization and deserialization ensure that the data remains consistent and uncorrupted during its journey from producer to consumer.

#### Custom Serializers and Deserializers

While Kafka provides built-in serializers and deserializers for basic data types (e.g., String, Integer, Bytes), users often need to implement custom Serializer and Deserializer interfaces to handle complex or custom data structures (e.g., POJOs, domain-specific objects). This involves defining how the object's state is converted to bytes and vice-versa.

#### **What can Kafka Monitoring be used to do?**

Kafka monitoring is used to track and manage the health, performance, and resource utilization of Apache Kafka clusters. It involves monitoring various metrics like throughput, latency, consumer lag, disk usage, and network activity to ensure smooth data flow and identify potential issues. By continuously analyzing these metrics, administrators can detect anomalies, optimize resource allocation, and prevent or resolve problems like replication issues, broker failures, or data inconsistencies.

#### **Name the various types of Kafka producer API**

#### **How can you write data from Kafka to a database?**

There are two different frameworks- Connect Source and Connect Sink. You can send topics from Kafka to external databases and load the data from source databases.



**A ZooKeeper ensemble** is a group of interconnected ZooKeeper servers that work together to provide a reliable and consistent service for distributed applications. It's a cluster of ZooKeeper servers that ensures high availability and fault tolerance for coordinating tasks across a distributed system.

Here's a more detailed explanation:

- **Purpose:**

ZooKeeper ensembles are designed to manage and coordinate distributed applications by providing a centralized service for configuration management, synchronization, and naming.

- **Structure:**

An ensemble consists of multiple ZooKeeper servers, typically an odd number (like 3, 5, or 7) to ensure a [quorum](#) can be maintained even in the event of server failures.

- [Quorum](#):

A quorum is a majority of servers in the ensemble. For example, in a 5-server ensemble, a quorum would be 3 servers. ZooKeeper requires a quorum to make updates to its data, ensuring consistency and reliability.

- [Leader](#) and Followers:

Within the ensemble, one server is elected as the leader, responsible for handling write operations. The other servers act as followers, replicating the leader's data and assisting in reads.

- **High Availability:**

By having multiple servers, the ensemble can tolerate the failure of some of its members. As long as a [quorum](#) of servers remains available, the ensemble can continue to operate and serve requests, ensuring high availability for the applications that rely on it.

- **Client Interaction:**

Clients connect to the ensemble to access ZooKeeper's services. They can read data, write data, and receive notifications about changes to the data.

- **Example:**

If a client wants to store a configuration value, it can write that value to the ZooKeeper ensemble. The ensemble will then replicate that value to a quorum of servers, ensuring that the information is available even if some servers fail.

## ZooKeeper, znodes

In ZooKeeper, znodes are data nodes within a hierarchical namespace that are similar to files and directories in a file system. They store data, have a name, and can contain child znodes, forming a tree-like structure. Each znode is tracked by a stat structure, which includes data, ACLs, version numbers, and timestamps.

### What are the types of Znodes?

There are three types of Znodes, namely:

**Persistence Znode:** these are the znodes that remain alive even after the client who created that particular znode is disconnected. All znodes are persistent by default unless otherwise specified.

**Ephemeral Znode:** Ephemeral znodes remain active only until the client is alive. Ephemeral Znodes get deleted whenever the client that created them gets disconnected from the ZooKeeper ensemble. They play an important role in the leader election.

**Sequential Znode:** when znodes are created, it is possible to request the ZooKeeper to add an increasing counter to the end of the path. This counter is unique to the parent znode. Sequential nodes may be persistent or ephemeral.

### What is Kafka's producer acknowledgment

In Apache Kafka, producer acknowledgments, controlled by the acks parameter, determine how a producer verifies message delivery to the broker. Producers can wait for the leader, all in-sync replicas, or no acknowledgment, impacting latency and data durability.

Here's a breakdown:

- acks=0:

The producer sends the message and doesn't wait for any acknowledgment. This offers the lowest latency but highest risk of data loss if the broker fails.

- acks=1:

The producer waits for acknowledgment from the leader replica. It provides a balance between latency and durability, but if the leader fails before replication, data loss is possible.

- acks=all (or -1):

The producer waits for acknowledgments from all in-sync replicas. This ensures high data durability and consistency but may increase latency.

**Can a Kafka Consumer group have more than one consumer?**

Yes, a Kafka consumer group consists of one or more consumers. In general, each consumer may associate with one partition. The additional Kafka consumers in the group become inactive if the number of consumers in the specific consumer group exceeds the number of partitions. So, you must always ensure that a Kafka